

Langage C

TP 10

Juliusz Chroboczek

4 avril 2026

Téléchargez le fichier `parser.tar.gz` fourni sur la page du cours et extrayez son contenu. Le code implémente un analyseur syntaxique (*parser*) pour la grammaire suivante :

$$F ::= \text{number} \mid \text{variable} \mid (E)$$
$$T ::= F * T \mid F / T$$
$$E ::= T + E \mid T - E$$

Par exemple, la chaîne « `2+3*pi` » obéit à cette grammaire, et conduit à l'arbre de syntaxe suivant :

```
      +
     / \
    2   *
      / \
     3  pi
```

Les fichiers `parser.c` et `lexer.c` implémentent l'analyseur syntaxique, vous n'avez pas besoin d'en comprendre le contenu. Le fichier `parser.h` définit l'interface de l'analyseur : elle consiste d'une fonction `getexpr`, qui analyse une expression et retourne l'arbre de syntaxe correspondant, et d'une fonction `free_expr` qui libère la mémoire utilisée par une expression. La fonction `getexpr` prend en paramètre une structure de type `struct lexer`, qui est construite à l'aide de `make_lexer` (défini dans `lexer.h`).

Exercice 1. Dessinez un diagramme boîtes-et-flèches qui décrit la représentation de l'arbre de syntaxe ci-dessus sous forme d'une `struct expr`.

Exercice 2.

1. Écrivez un `Makefile` et compilez les fichiers fournis. Testez le programme. Vérifiez à l'aide de *Valgrind* qu'il n'a pas de fuites de mémoire.
2. Écrivez une fonction `print_expr` de prototype

```
void print_expr(struct expr *expr);
```

qui affiche une expression en syntaxe infixe complètement parenthésée : pour l'expression ci-dessus, votre fonction affichera la chaîne « $(2 + (3 * \pi))$ ». Cette fonction procédera comme suit :

- si l'expression est un nombre, elle affiche sa valeur;
- si l'expression est une variable, elle affiche son nom;
- si l'expression est un opérateur binaire, elle
 - affiche une parenthèse ouvrante;
 - fait un appel récursif sur le premier opérande;
 - affiche l'opérateur (+, -, * ou /);
 - fait un appel récursif sur son deuxième opérande;
 - affiche une parenthèse fermante.

Ajoutez un appel à `print_expr` au `main` et testez votre programme. *Valgrind* a-t-il des objections?

3. Que se passe-t-il si vous passez à votre programme la concaténation de deux expressions, par exemple « $2+3 \ 3+2$ » ou « $2+3x$ »? Résolvez le problème en affichant un message d'erreur lorsque c'est le cas.

Exercice 3. Écrivez une fonction `eval_expr` de prototype

```
int eval_expr(struct expr *expr);
```

qui évalue une expression. Votre fonction retournera 0 pour une variable (on suppose que toutes les variables valent 0). Écrivez un programme qui lit une expression et affiche sa valeur. *Valgrind* a-t-il des objections?

Exercice 4. Écrivez une fonction `derive_expr` de prototype

```
struct expr *derive_expr(struct expr *expr, char *varname);
```

qui retourne une nouvelle expression qui représente la dérivée de l'expression `expr` par rapport à la variable nommée `varname`. Votre fonction construira une nouvelle expression, elle ne modifiera pas l'expression passée en paramètre.

Écrivez un programme qui lit une expression puis affiche sa dérivée par rapport à la variable `x`.

Exercice 5. Modifiez la fonction `print_expr` pour qu'elle n'affiche des parenthèses que lorsque c'est nécessaire. Par exemple, elle affichera « $2+3*4$ » pour l'arbre de syntaxe obtenu à partir de « $(2 + (3 * 4))$ », mais elle affichera « $(2+3)*4$ » pour l'arbre de syntaxe obtenu à partir de « $((2+3)*4)$ ». De même, elle affichera « $2-3-4$ » pour « $((2-3)-4)$ », mais « $2-(3-4)$ » pour « $(2-(3-4))$ ». (Indication : il faudra passer un paramètre supplémentaire à la fonction.)